有限位元中的算術契約

Computer arithmetic 是 finite-width representation 與 operation semantics。硬體加法器可以處理 bit patterns，但「候選結果超出可表示範圍」要由 signed interpretation、instruction 與 status flags 決定如何處理。

本章分三條線：integer ALU、multiply/divide state machine、IEEE 754 floating point。每條線都要指定 bit width、rounding、exception 與 timing model。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

算術章的問題

固定 32-bit word 無法表示所有整數、分數或實數。Overflow、underflow、rounding 與特殊值不是書寫誤差，而是 representation 契約的一部分。

硬體設計要平衡 latency、throughput、面積與 exception precision。Fast multiplier 可以用更多 partial-product hardware 換取較少 iterations；IEEE add 可用 extra bits 降低 rounding error，但增加 pipeline state。

本章先定義 operation result，再設計 hardware。不能只從 gate diagram 反推語意。

例如 8-bit unsigned 乘法的 operands 各 8 bits，完整 result 需 16 bits；若只存低 8 bits，即明定取 modulo 256。Binary32 加法則輸入 24-bit 有效精度的值，先形成 exact sum，再在規定 rounding point 映回 binary32；兩種 result 規則不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 算術章的基本模型

Integer arithmetic 在 modulo 2^n ring 中進行，除非 instruction 選擇 trap 或 saturated result。Floating point 則在有限 set F 上操作，先依 exact 或 extended-precision rule 計算，再把無法精確表示的結果映射到 F。

對任何 operation $\circ$，硬體要給出

$$y = \mathrm{round}(x \circ z),$$

其中 rounding function、exception conditions 與 special cases 都必須明確。

Floating point 與 real arithmetic 不同。運算結果滿足 ISA 定義的 correctly rounded behavior，不代表在所有平台上都得到无限精確的數學結果。

整數 `0/0` 的處理依 ISA 或語言而異；IEEE 浮點 `0/0` 為 invalid，通常產生 quiet NaN。先指定型別與架構再談除法。

3.2 二補數加減

對 n-bit two's complement integers，bit vector 可解釋為

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

Addition 的 bit-level result 與 unsigned addition 相同，輸出低 n bits。Signed value 是否等於 mathematical sum 取決於是否 overflow。

Subtraction 可寫成 $a - b = a + \bar{b} + 1$，因此可用相同 adder 與 $carry_in = 1$。此代數關係是硬體共用 adder 的基礎。

Modular bit pattern 正確不等於 signed mathematical result 可表示。兩者要分開陳述。

Signed overflow 的推導

令 operands 與 result 的 sign bits 分別為 s_a, s_b, s_r。Addition 只在 operands 同號且 result 異號時 signed overflow：

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

等價地，carry into MSB 與 carry out of MSB 不同，也代表 signed overflow。

Unsigned overflow 則只由 carry-out 表示。比較 $0xffffffff$ 與 1 時，unsigned 是最大值對 1；signed 是 -1 對 1。

用 `0x7fffffff+1` 和 `0xffffffff+1` 分別檢驗 signed 與 unsigned 條件。兩者不可共用一個「overflow 等於 carry」的句子。

Addition/subtraction 的位元級驗算

令 c_7 是進 sign bit 的 carry，c_8 是離開 sign bit 的 carry。對 8-bit addition，signed overflow 等價於

$$ov = c_7 \oplus c_8.$$

Signed overflow 的推導 （續）

Operation	Operand bits	Result bits	c_7	c_8	Unsigned carry-out	Signed overflow
`0x7f+0x01`	`01111111+00000001`	`10000000`	1	0	0	1
`0x80+0xff`	`10000000+11111111`	`01111111`	0	1	1	1
`0x80-0x01`	`10000000+11111110+1` 的減法模型	`01111111`	0	1	1	1
`0xff+0x01`	`11111111+00000001`	`00000000`	1	1	1	0

`0xff+1` 的 $c_7 = c_8 = 1$，只表示 unsigned carry-out。相同結果 bits 若由 `slt/sltu` 解讀，大小順序也不同。減法可改寫為 $a - b = a + \bar{b} + 1$；對 subtraction，$ov = (s_a \oplus s_b) \wedge (s_a \oplus s_r)$。

加減法時序與例外

Simple ripple-carry adder 的 carry 依序傳遞，critical path 隨 width 成長。Carry-lookahead 與其他 parallel-prefix 結構用更多 gates 縮短 carry path，但增加面積與 wiring。

若 instruction 是 `add`，signed overflow 可能寫入例外原因並阻止 architectural result commit。`addu` 採 modulo result，不因同樣條件 trap。具體的 exception 與 register side effect 以 ISA 定義為準。

Timing 必須以最壞可成立 path 與 setup/hold constraints 決定，不能只用平均輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多媒體飽和運算

Saturated arithmetic 在超出 representable range 時把結果夾到最小或最大值。例如 signed 8-bit 的 $100 + 100$ 飽和結果是 127，而不是 modular bit pattern。

Saturation 避免 wrap，但會損失可逆性。Encryption、checksum 或需要 exact modulo 的程式不適合任意改用飽和。

SIMD packed arithmetic 可同時處理多個短 integers。Carry 不跨 lane，lane width 與 saturation mode 是 instruction 契約的一部分。

同為 signed 8-bit 的 100+100，audio saturation 夾為 127；保留低 8 bits 則為 0xc8，作 two’s-complement 解讀是 −56。若以 unsigned byte 解讀 0xc8 則是 200。選擇 saturate、wrap 及 interpretation 會改變可觀察結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 乘法硬體

Unsigned $n \times n$ multiplication 的 exact product 最多需 $2n$ bits。32-bit operands 的 exact result 是 64 bits，結果不應先截成 32 bits 再計算。

Shift-and-add 依據

$$A \times B = \sum_{k=0}^{n-1} b_k (A \ll k).$$

若 $b_k = 1$，累加 shifted multiplicand；若 $b_k = 0$，不需要累加。乘 2 的冪可由移位完成，但生成一般 partial products 仍需 AND gates。

先固定是 signed 還是 unsigned multiplication。相同 bit pattern 在兩種 interpretation 會得到不同 mathematical value。

循序乘法時序與正確性

對 unsigned n-bit M、Q，每回合只看 Q_0，需要時計算

$$C : A \leftarrow A + M,$$

再把 C, A, Q 串成 $2n+1$ bits 右移一位；C 進入 A 的 MSB，舊 Q_0 已處理並移出；舊 Q_1 移入新的 Q_0，才是下回合 decision bit。下表以 $M = 1101_2 = 13$、$Q = 1011_2 = 11$ 明定 A 為 shift 後的 4-bit 值，C 只表示該回合加法的第五個 carry bit。

Round	Q_0	Add result $C:A$	Shift 後 A	Shift 後 Q
0			`0000`	`1011`
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

循序乘法時序與正確性 (續)

最終 (A,Q)=10001111₂=143=13×11。要證明右移程序，固定初始multiplier為 q，完成 k 回合的暫存器為 A_k, Q_k，定義 $R_k = 2^n A_k + Q_k$。精確不變量是

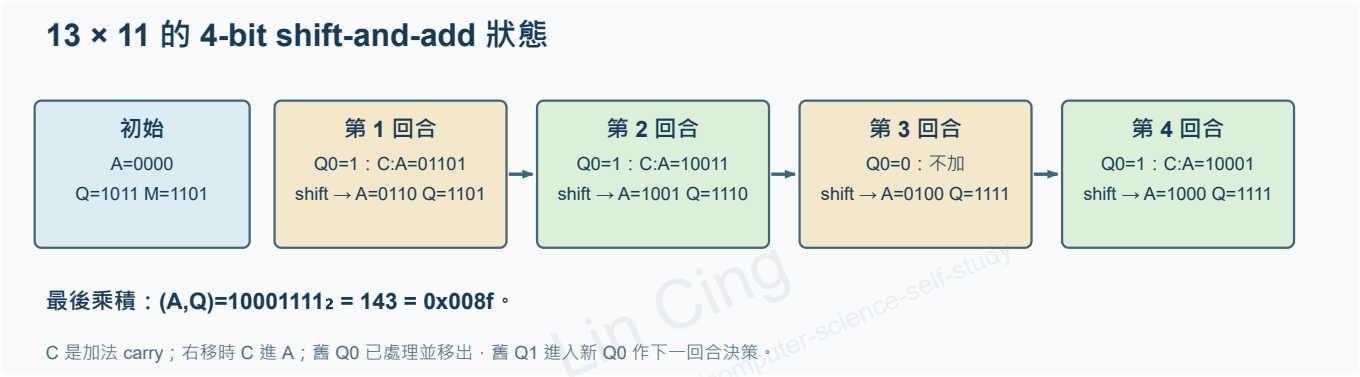
$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)\, 2^{n-k}, \qquad 0 \le k \le n.$$

$k = 0$ 時 $R_0 = q$，成立。若 $k < n$，右項第二項為偶數，所以當前 Q_k 的最低位恰為原multiplier第 k 位 q_k。依此位決定加法後，完整 $C : A : Q$ 的值為 $R_k + q_k M 2^n$；右移得到

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1}) 2^{n-k-1}.$$

這使用 $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$，因此歸納延續。$k = n$ 時因 $q < 2^n$，有 $R_n = Mq$；A為乘積高半部，Q為低半部。Carry C保留本回合加法的溢位並隨右移進A，沒有遺失該位。13×11的實際 R_k 依序為 11、109、158、79、143。

循序乘法時序與正確性 (續)



成本模型先寫清楚才可比較：

$$T_{\text{serial}} = t_{\text{setup}} + n\, t_{\text{iter}} + t_{\text{finalize}}.$$

若每回合的 add 與 shift 能同 cycle 完成，32-bit model 是 32 iterations；若它們各需一個 cycle，不能仍宣稱固定 32 cycles。Parallel tree 減少 dependent additions，但增加 partial-product wires、compressors 與 power；pipelined multiplier 可縮小 initiation interval，不會縮短單筆 latency。

有號乘法

一種方法是記錄 operands 的 signs，取 magnitudes 作 unsigned multiplication，再依 sign product 決定結果符號。

對 n-bit signed operands，exact product 的 signed range 需要 $2n$ bits。最極端負值平方可能需要完整的 $2n$-bit pattern。

另一種硬體方法使用 Booth recoding，直接從 two's complement bits 生成 signed partial products，避免先取 absolute value。Radix-4 Booth 每次看多個 bits，減少 partial products，但 control 與 sign extension 更複雜。

Lin Cing
https://github.com/xinhi6716/computer-science-self-...

乘法加速

Parallel multiplier 一次產生多個 partial products，再用 Wallace tree、compression tree 或 carry-save adders 壓縮。Carry-save 可避免每一階段都傳遞長 carry chain。

Final carry-propagate adder 通常在最後產生 non-redundant result。樹狀結構降低 latency，代價是較多 full adders、routing 與 power。

Pipeline multiplier 可接受新 operand 的間隔比單次結果 latency 更短。Throughput 與 latency 是不同指標。

如果每 cycle 發一個 multiply 但結果三 cycles 後才可用，需 register scoreboard 或 hazard detection。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIPS 乘法指令

MIPS 使用 `mult`、`multu` 把 product 放入 64-bit HI/LO pair。`mfhi`、`mflo` 把指定 half 搬到 general-purpose register。

Unsigned 與 signed instruction 的 bit-level hardware 可共用，但高半部的數值意義不同；傳統 MIPS32 `mult`/`multu` 不因乘積超出 32 bits 而 trap，兩者都產生完整 HI/LO 結果。

若程式只取 32-bit low product，high bits 的 overflow 可能未被檢查。決定 result 是否精確要看成員需求與 instruction semantics。

不要把 `multu` 解讀為 operands 永遠是正數；它是 unsigned interpretation of bit patterns。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 除法硬體

Division 求 quotient q 與 remainder r，使

$$D = Q \times d + r, \qquad 0 \le r < |d|.$$

這個範圍版本適用 unsigned division。Signed division 還要指定 quotient 的 truncation direction 與 remainder sign；不同語言、ISA 的代數約定可能不同。

Divide by zero 與 `INT_MIN / -1` 的結果都必須由具體 ISA 定義。傳統 MIPS32 `div`/`divu` 不因除以零或有號除法溢位而產生算術 trap；這些情形的 HI/LO 結果不可依賴，程式需先檢查。其他 ISA 或語言可能定義 trap，不能混用。

四條必要檢查是 divisor zero、sign policy、rounding direction、minimum-value boundary。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restoring division algorithm

Unsigned restoring algorithm 每回合左移 rem 與 dividend，把 divisor 從 rem 中減去。若非負，quotient bit 設 1；若為負，restore rem 並設 quotient bit 0。

用 29 除以 6，5-bit unsigned 教學模型：

1. rem 0，dividend quotient register 從 `11101` 開始。
2. 逐位產生 quotient。正確結果為 $Q = 4$、$R = 5$，因為 $29 = 4 \times 6 + 5$。
3. 32-bit 硬體需類似 33-bit remainder width 或額外 sign bit，才能保留 subtraction 是否成功的資訊。

Restoring division 的逐位不變量

用 bitwise long division 描述較容易證明。第 k 回合開始時，令已產生 quotient bits 為 Q_k、目前 remainder 為 A_k，並維持

$$P_k = Q_k M + A_k,$$

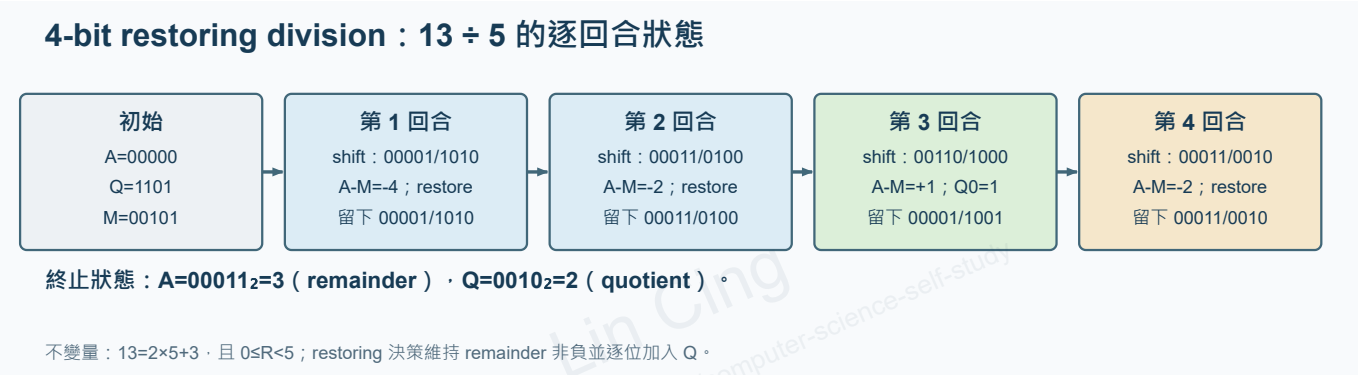
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restoring division algorithm (續)

其中 P_k 是已讀入的 dividend prefix。下一 dividend bit d 到來後，候選 remainder 是 $2A_k + d$。若它至少 M，本回合 quotient bit 放 1，remainder 改為 $2A_k + d - M$；否則放 0，remainder 保持 $2A_k + d$。兩個分支都維持上一式，因此做完所有 dividend bits 後 $D = QM + A$，且 $0 \le A < M$。這就是 restoring division 的正確性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restoring division algorithm （續）



對 $D = 13$、$M = 5$，最終 $A = 3$、$Q = 2$。四個 shifts 後驗證 $13 = 2 \times 5 + 3$。這個 combined-register variant 有 n rounds；書上另一個不提前折疊 register 的 hardware variant 可能寫成 $n + 1$ rounds，兩者 iteration 數不能混用，卻必須得到同一 pair。

Signed division 與 faster division

取 magnitudes 後作 unsigned division，再依 signs 決定 quotient sign。Remainder sign 通常跟 dividend 一致，因為 $D = Qd + R$；實際規則仍依 ISA 或 programming language 定義。

SRT divider 每回合用 estimate 與 residual recurrence 產生多個 quotient bits。Radix-4 每回合可產生 2 bits，重複次數約為 binary radix 的一半，但需要 quotient-digit selection table。

對極端 divisor、leading-zero 分佈與 bit pattern 的高度偏差，可做 operand normalization 與 early termination。這屬實作最佳化，不可改變 architecture-visible result。

不要把 radix-4 的「2 bits per iteration」直接當成精確 2 倍 speedup。Overhead、normalization 與 correction cycles 仍存在。

Divide 指令

教材的 MIPS 使用 `div`、`divu`，結果分布在 HI/LO。程式再用 `mfhi` 取 remainder、`mflo` 取 quotient。

Divide latency 常較 multiply 長，且 fewer iterations 不代表 pipeline throughput 同比例改善。若 compiler 看到除數是常數，可能用 multiply by reciprocal 或 shifts，但必須證明 rounding semantics 等價。

Optimization 只在輸入範圍與 compiler 支援的 rounding model 相符時有效。

以 two's-complement `x=-7` 為例，MIPS `sra` 右移1得到 -4（朝負無窮），而 truncation-toward-zero 的有號除以2得到 -3。對此一位右移可先為負數加1，再 `sra` 得 -3；一般除以2的k次方須先證明適當 bias、位寬及不溢出，不能把任意 shift 直接當除法。

3.5 IEEE 754

IEEE 754 binary floating-point 以 sign、biased exponent 與 fraction 表示

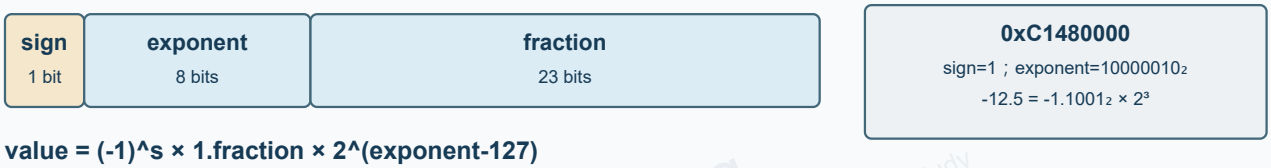
$$value = (-1)^s \times \text{significand} \times 2^e.$$

Normal binary32 有 1 sign bit、8 exponent bits、23 fraction bits。Exponent field 1 至 254 對應 biased exponent，bias 為 127，leading 1 隱含在 significand。

Binary64 有 1、11、52 bits，bias 為 1023。Exponent width 和 significand precision 不能互相替代。

浮點欄位

IEEE 754 binary32 欄位與 -12.5 編碼



0xC1480000

sign=1；exponent=10000010₂

-12.5 = -1.1001₂ × 2³

value = (-1)^s × 1.fraction × 2^(exponent-127)

Exponent 全 0 改用 subnormal 規則；全 1 表示 infinity 或 NaN，不能一律套 hidden 1。

Normal number 的 value 可寫成

$$(-1)^s (1.F)_2 \, 2^{E_{\text{field}} - bias}.$$

其中 F 是 fraction bits。Single 的 significand precision 是 $23 + 1 = 24$ bits，double 是 $52 + 1 = 53$ bits。

浮點欄位 （續）

Denormal/subnormal 在 exponent field 為零時使用，leading bit 改為 0，single 的 implicit exponent 是 -126。它讓零附近保持 gradual underflow。

`hidden 1` 不是永遠存在。Exponent field 為 zero 時必須切換 significand rule。

特殊值與例外

Exponent field 全為 1 時表示 infinity 或 NaN。Fraction 為 0 是 signed infinity；fraction 非 0 是 NaN。NaN 又分 quiet 與 signaling，payload 與 propagation 有 IEEE 規則。

Operations 可能設定 overflow、underflow、inexact、invalid 與 divide-by-zero flags。Trap 是否啟動由 control register 與程式語言環境決定。

$+\infty + (-\infty)$ 是 invalid operation，通常產生 quiet NaN。$1/+\infty$ 在 IEEE arithmetic 中可以是 $+0$，與 integer divide by zero 不同。

不要用程式語言的 exception 名稱替代 IEEE flag 名稱；兩者的 trap 機制與 observable 行為可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

浮點加法

對 $a + b$，典型步驟如下。

1. 檢查 NaN、infinity 與 zero special cases。
2. 比較 exponentials，把較小者 right-shift 以對齊 radix point。
3. 加減 significands，保留 extra bits。
4. Normalize result。
5. 依 rounding mode 捨入，必要時再 normalize。
6. 偵測 overflow、underflow 與 inexact。

Alignment shift 可能把 tiny operand 的部分或全部 bits 推出可影響範圍，這時 guard、round 與 sticky bits 保存 decision 所需資訊。

若先任意截成相同 precision 再相加，可能改變 tie case。Sticky bit 表示被丟棄的位元是否任一個非零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Guard、round 與 sticky

考慮 sign-and-magnitude significand。Guard bit 是 rounding position 右邊第一位，round bit 是第二位，sticky 記錄更右側是否曾有 1。

Round-to-nearest-even 判斷大於、小於或正好一半，tie 時選最低保留 bit 為 0 的 candidate。

Fused multiply-add（FMA）先求精確 $ab + c$ 再捨入一次；分開的 multiply 與 add 在乘積與和各捨入，兩者不保證 bitwise 相同。是否存在 FMA 指令依 ISA 版本而定，不能預設本章舊版 MIPS 都支援。

以 binary32 計算 $1 + 2^{-24}$：

- exact 結果是 $1.00000000000000000000000001_2$ 與下一 representable value 的一半。
- tie 的兩個 candidates 為 1 與 $1 + 2^{-23}$。
- 保留 bit 的最低位在 1 為 0，在另一 candidate 為 1，所以選 1.0。

每個 operation 的 rounding point 都是語意的一部分。例如 binary32 的 `x=2^24` 與 `x+1` 相加正好落在兩個 representable values 的中點，RNE回到x；再減x得到0，而先算x−x再加1得到1。

Guard、round 與 sticky （續）

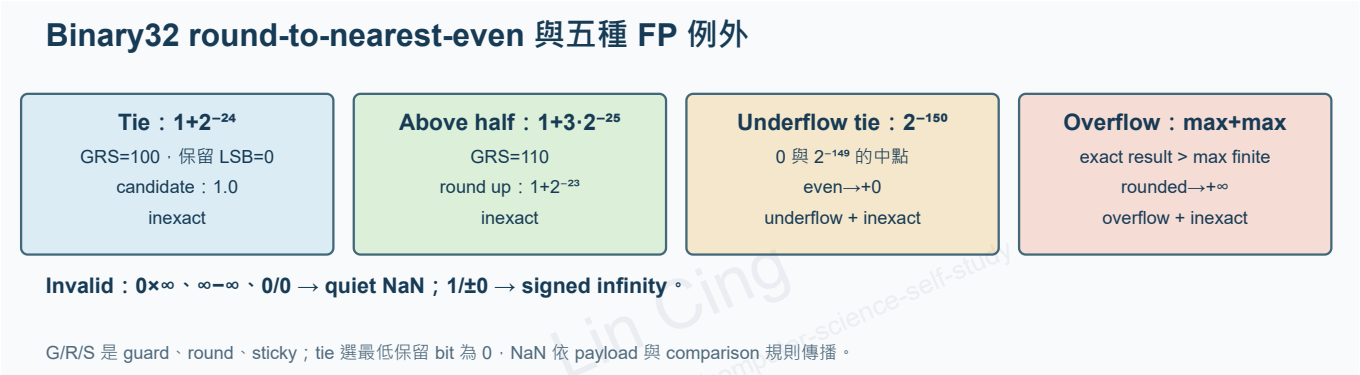
Binary32 rounding、特殊值與例外

Rounding 的輸入必須是一組 exact retained bits 加上 G, R, S。令最低保留 bit 為 L。Round-to-nearest-even 的三個條件是：

1. $G = 0$：丟棄部分小於一半，向下保留。
2. $G = 1$ 且 $R = 0, S = 0$：剛好 halfway，選 $L = 0$ 的 candidate。
3. 其他 $G = 1$ 情況：大於一半，向上加一。

以 $1 + 2^{-24}$ 為例，保留 $1.000\ldots000$，丟棄 bits 是 `100`，所以 tie 且 $L = 0$，結果回到 `0x3f800000`。以 $1 + 3 \cdot 2^{-25}$ 為例，丟棄 bits 是 `110`，大於一半，結果升到 $1 + 2^{-23}$。

Guard、round 與 sticky （續）



Guard、round 與 sticky （續）

對最小 subnormal 2^{-149} 的一半 2^{-150}，兩個候選是 $+0$ 與 2^{-149}；tie 選 even 的 $+0$，因此同時記錄 underflow 與 inexact。對最大 finite 加自身，exact value 超過可表示上界，default RNE 得到 $+\infty$，記錄 overflow 與 inexact。`0×∞`、`∞−∞` 是 invalid；視情況產生 quiet NaN。有限且非零的分子除以±0，結果的符號為兩operand sign bits的XOR，產生signed infinity並記錄divide-by-zero。∞/+0仍為相同符號的∞，不因這次操作設定divide-by-zero；0/0則屬invalid。這些旗標判準見Oracle Numerical Computation Guide表31。

NaN 不會因比對就變成普通 number。Ordered compare 在任一 operand 是 NaN 時失敗；unordered compare 才可用來偵測「含 NaN」。`+0` 與 `-0` 的 equality 相同，但 reciprocal、signbit 與 serialization 可觀察到差異。

浮點乘法

對 $a = (-1)^{s_a} 1.F_a 2^{e_a}$、$b = (-1)^{s_b} 1.F_b 2^{e_b}$，exact significand product 是

$$(1.F_a)(1.F_b),$$

result exponent 概念上是 $e_a + e_b$。Product 可落在 $[1, 4)$，需 normalize，再捨入到 target precision。

Sign 由 $s_a \oplus s_b$ 決定。Subnormal operands 可能需先 normalize 成有效 precision，或因 tiny result 進入 underflow path。

Zero multiplication 要處理 signed zero 與 infinity combinations。$0 \times \infty$ 是 invalid operation，不是一般 zero。

乘法比加法少 alignment，但 significand multiplier 可能較寬，仍需 extra precision 與 final rounding。

MIPS 浮點指令

本章案例 MIPS floating-point 使用 coprocessor 1 registers，例如 `lwc1`、`swc1` 搬移 floating-point words，`add.s`、`add.d` 區分 formats。

Floating-point comparison 設定 condition code，branch 透過 `bc1t`、`bc1f` 等指令檢查。NaN 的 ordered/unordered comparison 需要特別 semantics。

Register number 空間與 general-purpose registers 分開。Compiler 要避免 format 混用並計算 save area 大小。

本節是教材 MIPS 案例。不要把 MIPS FP encoding 直接套用到 RISC-V 或 Arm。

精確算術與捨入模式

Round-to-nearest-even 是常用 default，因為它降低 bias。Directed modes 可往正 infinity、負 infinity 或 zero 捨入，適合 interval arithmetic 或 precise bounds。

ULP 是相鄰 representable numbers 的間距。絕對 error 小不代表 relative error 小；在很大 exponent 下，ULP 也可以很大。

Floating-point format 的 precision 是 significand bits，dynamic range 由 exponent bits 決定。增加 exponent 不增加 precision，增加 fraction 才增加 precision。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NaN、signed zero 與比較

NaN 不與任何 value 成立 ordinary ordered equality，包含自己。Program 可用 `isnan` 或 unordered comparison 測量。

$+0 = -0$ 在 IEEE equality 中成立，但兩者 sign bit 不同，可能在 reciprocal、copy sign 與 serialization 中顯現。

Infinity arithmetic 有部分代數規則。$\infty - \infty$、$0 \times \infty$、∞ / ∞ 是 invalid，結果通常為 quiet NaN。

比較浮點值不能只用 signed integer bit order，因為 negative numbers 的 encoding 與 NaN。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 平行與結合律

Exact integer addition 的 associativity 在 modular arithmetic 中也成立，但 floating-point rounding 使

$$(a+b)+c \neq a+(b+c)$$

可能成立。即使 exact sum 相同，rounding point 不同也會改變 result。

Parallel reduction 若改變 summation tree，結果可能因 rounding 改變。這不是 race，而是 numerical reproducibility 問題。

若要求bitwise reproducibility，需固定求和次序及相同浮點環境（格式、捨入模式、中間精度與FMA收縮等），或使用具明確順序無關保證的reproducible accumulator。Compensated summation與提高precision主要改善誤差，本身不保證換序後位元相同。

標準Kahan若每次基本運算都以binary32的round-to-nearest-even執行，順序 $[2^{25}, 1, -2^{25}]$ 得0，改成 $[2^{25}, -2^{25}, 1]$ 得1。第一順序中加1時sum仍為 2^{25}、補償c變−1；接著 $-2^{25}+1$ 位於中點，RNE又捨回 -2^{25}，總和成0。第二順序先精確抵銷成0，再加1得1。補償降低許多累加誤差，卻沒有消除對運算次序與環境的依賴。

3.6 平行與結合律 （續）

同一個程式在不同 thread scheduling 下得到略不同答案，不一定是 memory corruption；先檢查 reduction order。

FMA 與兩次 rounding 的差別

浮點 add 的特殊 grouping 也要用 bit pattern 驗證，而不是宣稱數學上相等。以 binary32 $x = 2^{24}$ 為例，x 的 ULP 是 2。x+1 恰好落在 x 與 x+2 的中點，最低保留 bit 要求選 even，所以先 round 成 x：

$$((x+1)-x) = 0, \qquad (x-x)+1 = 1.$$

兩式在 exact real arithmetic 相等，在逐次 binary32 rounding 下不相等。這是 rounding point 改變，不是 data race。

令 $a = b = 1 + 2^{-23}$（`0x3f800001`）、$c = -(1+2^{-22})$（`0xbf800002`）。分開執行 `mul.s` 後再 `add.s`：

$$(1+2^{-23})^2 = 1 + 2^{-22} + 2^{-46},$$

3.6 平行與結合律 （續）

乘積先 round 成 $1 + 2^{-22}$，再加 c 得 `+0`。若假設的 fused multiply-add（FMA）依 IEEE semantics 只 round 一次，exact $ab + c = 2^{-46}$ 可直接表示為 `0x28800000`。這不是因為 FMA 改變 real algebra，而是少了 intermediate `1+2^{-22}` rounding。

本章舊版 MIPS core 不保證有 FMA；有 `fma` 的 extension 必須另外指定 rounding mode 與 operand format。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7 x86 浮點案例

教材比較 x87 與 SSE2。x87 以 internal extended-precision stack registers 執行，SSE2 使用 XMM registers 作 SIMD packed/scalar operations。

把 value 存在寬 internal precision，再 round 到 memory format，與每一步都 round 到 target precision，可能得到不同結果。ABI 與 compiler flags 會改變 intermediate precision。

SSE2 支援 packed multiply/add 等 instructions，可提高資料層級平行。Instruction form、alignment 與 lane semantics 必須符合實際 ISA。

歷史 x87 與今日 AVX 不可混用。Version、OS context 與 compiler `fast-math` 都要記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.8 算術誤解

「Floating point 只是近似，所以不用管」忽略近似方式本身有嚴格規則。IEEE result、exception flags 與 rounding mode 能被應用精確依賴。

「Fast-math 只會提高速度」忽略 compiler 可重排、假設無 NaN 或改變 associativity。安全性與 reproducibility 可能受影響。

「Division 一定比 multiplication 慢固定倍數」忽略 operand-dependent divider、pipeline、specialization 與 instruction mix。

每條誤解都要求一個可驗證的基準與反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.9 算術小結

Integer operations 的 width、overflow 與 modulo semantics 要一起看。Multiply/divide 的 iteration 可能是 32 次數量級，但實際 cycles 依 radix、normalization 與 pipeline 不同。

Floating-point unit 是三個 functions 的組合：representation、arithmetic、rounding/exception。Add 的關鍵是 alignment 與 lost bits；multiply 的關鍵是 significand product 與 normalization。

精確性驗收應包含 normal、zero、subnormal、infinity、NaN、overflow、underflow 與 tie-to-even。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數值格式演進

早期 machines 使用不同 exponent width、bias 與 signed representations。IEEE 754 標準化 binary formats、operation rounding 與 special values，使程式能跨硬體得到可預測結果。

硬體從 serial add/shift multiplier 走向 radix recoding、parallel partial-product tree、pipeline FP unit 與 SIMD。Instruction 與 compiler 也共同把更多算術工作放到 vector lanes。

演進沒有消除數值分析。大動態範圍、ill-conditioned problems 與 reproducible parallelism 仍需 algorithm-level 處理。

將 format、arithmetic hardware、rounding 與 algorithm 分成四層，不要用「IEEE 已解決浮點」概括。

章末驗收

學生要能以 bit-level trace 完成以下工作。

1. 判定 signed/unsigned addition overflow，並檢查 carry-in/carry-out 與 sign-bit 條件。
2. 用 4-bit 教學模型逐回合追蹤 multiply 與 restore division，說明 register width、carry 與 correctness invariant。
3. 編碼與解碼 binary32，正確處理 subnormal、infinity、NaN 與 signed zero。
4. 在 alignment 後使用 guard/round/sticky 完成 FP add，並比較 tie-to-even 與 above-half 的 rounding。
5. 對 overflow、underflow、invalid、divide-by-zero 與 inexact 給出 result 和 flag。
6. 證明 exact-equal algebraic expressions 在 floating point 不一定得到同一 bit pattern。
7. 比較 FMA 與 multiply-then-add 的 rounding point，求出可觀察差異的 bit patterns。

參考資料

Patterson 與 Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，3.1–3.10，印刷頁 222–283（PDF 頁 249–310）。

整數乘法、restoring division 與 multi-step floating-point operation 的成本以本文宣告的教學模型計算，不代表特定商用 core 的固定 latency。IEEE 754 binary32 special values、rounding modes、guard/round/sticky 與 FMA single-rounding 條件均需與 operand representation 一起解讀。

完整書目、版本與未交付範圍見 章節導覽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study